

 CS-2050 / project 



 **Code**  Issues  Pull requests  Actions  Projects  Security  Insights

Attention users: code.harvard.edu will be unavailable on 04/18/2026 from 9:00AM to 3:00PM for maintenance update.



Use this template

☆ 0 stars  0 forks  1 watching  1 Branch  0 Tags  Activity  Custom properties

 Internal template repository

 main ▾  1 Branch  0 Tags  

 Go to file 

Go to file

Code

...



wcw398 update.

5 days ago



README.md

update.

5 days ago

README



CS 2050 Final Project

Deadline: **May 7, end of day.** Projects submitted late will lose 20% of the total available points, per day.

Overview

The final project requires you to apply the high-performance computing techniques learned in class to an algorithm or other computational task of your choice. The project will be completed **individually**. You will begin with a serial version of your algorithm, and then develop parallel versions using OpenMP, MPI, and CUDA. You will analyze its performance in scaling as well as with profiling tools. Finally, you will briefly explore an additional programming paradigm of your choice. You will report all results in a blog-style report.

Important

Your task/algorithm does not need to be extremely complex. A simple algorithm is sufficient, provided it is accompanied with careful implementations and thoughtful analysis. It is far more important to complete all the steps successfully than to have an elaborate algorithm. We expect the total project effort to be *less than or equal to two homework assignments*.

Use of external resources (including AI tools) is encouraged; however, each student is expected to understand every detail of their submission.

Project Components

Part 1: Serial Implementation [10 Points]

Implement in C++ a serial version of your chosen algorithm to serve as the performance baseline. Measure runtime carefully. Include a Slurm script that compiles and runs your code to illustrate its use.

Part 2: OpenMP Implementation [10 Points]

Parallelize your algorithm using OpenMP for shared-memory execution. Evaluate the performance across varying thread counts, and verify correctness with appropriate tests. Include a Slurm script that compiles and runs your code to illustrate its use.

Part 3: MPI Implementation [10 Points]

Develop a C++ distributed-memory version using MPI. Design and analyze an appropriate communication strategy, and verify correctness with appropriate tests. Include a Slurm script that compiles and runs your code to illustrate its use.

Part 4: CUDA Implementation [10 Points]

Implement a GPU-accelerated version of your algorithm using CUDA. You may use any of the flavors of CUDA discussed during the course (including Python versions). Verify correctness with appropriate tests. Include a Slurm script that compiles and runs your code to illustrate its use.

Part 5: Additional Implementation [10 Points]

Extend your project by exploring an additional language or framework not covered in Parts 1-4 (e.g., mpi4py, Python multiprocessing, Julia, PyTorch, JAX, Kokkos, or others). Compare performance, ease of development, and abstraction level against your previous implementations where applicable. Include a Slurm script that compiles (if applicable) and runs your code to illustrate its use.

Part 6: Report [30 Points]

Write a blog-style report summarizing your project. It should include an introduction to your algorithm, as well as methods, results, and conclusion sections. Explain your parallelization strategies and key performance results, including figures to showcase scaling (both strong and weak scaling) and further explain your ideas. Use at least one profiling tool (VTune for CPU, Nsight Systems or Nsight Compute for GPU) to identify bottlenecks or explain observed performance trends.

We expect roughly 2500 words, but the exact word count is much less important than overall quality.

Part 7: Professionalism [20 Points]

How creative and ambitious is the project? Are the deliverables organized and of high quality? Have you followed all directions?

Submission Format

Projects will be submitted following the same process used for Homework submissions.

You will create your own copy of the provided GitHub template repository. All code and written components must be committed and pushed to your repository. Final submission will consist of a small PDF uploaded to Gradescope that identifies your final commit.

Your repository must follow this structure:

```
final-project/
├── README.md
├── report
│   ├── report.md
│   └── supporting figures, plots, etc
├── serial/
│   ├── source files
│   ├── Slurm scripts that reproduce all results
│   └── results, plots, etc
├── openmp/
│   ├── source files
│   ├── Slurm scripts that reproduce all results
│   └── results, plots, etc
├── mpi/
│   ├── source files
│   ├── Slurm scripts that reproduce all results
│   └── results, plots, etc
├── cuda/
│   ├── source files
│   ├── Slurm scripts that reproduce all results
│   └── results, plots, etc
├── additional/
│   ├── source files
│   ├── Slurm scripts that reproduce all results
│   └── results, plots, etc
```



All directories must be used appropriately. Code should be clean and organized. Plots and results should be reproducible.

Create your own copy of this repository